

FastAPI 依赖注入_介绍



🔍 什么是依赖注入？

依赖注入 (DI) 是一种设计模式，允许我们将“功能依赖”以声明式的方式注入到路由、中间件或其他组件中。FastAPI 内建了强大的依赖注入系统，它不仅支持函数依赖，还支持类依赖、嵌套依赖等高级用法。

✅ 优势：

- 提高代码复用性
- 解耦业务逻辑与通用逻辑（如认证、数据库连接）
- 支持异步依赖
- 自动解析类型提示和参数

依赖注入的作用域

FastAPI 依赖注入主要有三个级别,区别在于作用域不同：

- 1 **路径(Path Operation) 级别**：最常用，注入到 `@app.get()` / `@router.post()` 等装饰器下面的函数中
- 2 **路由(APIRouter) 级别- 共享依赖**：将依赖注入到整个路由器下的所有路径操作
- 3 **全局(FastAPI) 级别-全局依赖**：将依赖注入到整个应用的所有路径操作

💡 Depends() 基本语法

依赖可以是简单的函数或者类，FastAPI 会自动解析并执行。

示例代码

```
1 from fastapi import FastAPI, Depends
2
3 app = FastAPI()
4
5 # 简单依赖函数
6 def get_query_param(q: str = "default"):
7     return q.upper()
8
9 # 使用依赖的路由
10 @app.get("/items/")
11 async def read_items(query: str =
12     Depends(get_query_param)):
13     return {"query": query}
```

说明

- `get_query_param` 是一个依赖函数，接收查询参数 `q` 并返回其大写形式。
- 在 `read_items` 中，`query` 通过 `Depends(get_query_param)` 注入。
- FastAPI 自动处理参数提取和验证。

测试

访问 `/items/?q=hello` → 响应: `{"query": "HELLO"}`

如果未提供 `q`，则默认返回 `"DEFAULT"`。

```
1 from fastapi import FastAPI, Depends, HTTPException
2
3 app = FastAPI()
4
5 # 定义一个依赖函数
6 def common_params(q: str = None, skip: int = 0, limit: int = 10):
7     return {"q": q, "skip": skip, "limit": limit}
8
9 @app.get("/items/")
10 async def read_items(params: dict = Depends(common_params)):
11     return params
```

测试

访问 `/items/?q=hello&skip=2&limit=5` 会返回:

```
1 {"q": "hello",
2  "skip": 2,
3  "limit": 5}
```

说明

- 1 请求到达 `/items/`
- 2 FastAPI 发现 `params: dict = Depends(common_params)`
- 3 调用 `common_params(q=..., skip=..., limit=...)` 并传入当前请求参数
- 4 将返回值注入到 `read_items` 函数中作为 `params`

实时效果反馈

1. FastAPI 中的依赖注入（DI）的主要优势不包括以下哪一项？

- ☐ A 提高代码复用性
- ☐ B 解耦业务逻辑与通用逻辑
- ☐ C 支持异步依赖
- ☐ D 强制使用全局变量

2. 在 FastAPI 中，依赖注入的作用域不包括以下哪一项？

- ☐ A 路径（Path Operation）级别
- ☐ B 路由（APIRouter）级别
- ☐ C 全局（FastAPI）级别
- ☐ D 数据库级别

答案

1=>D 2=>D

FastAPI 依赖注入-路径级



在 FastAPI 中，**路径级依赖注入**是最常用的方式，它允许我们在单个路由中注入依赖项。

这些依赖项会在请求到达路由处理函数之前执行，并可以用于参数验证、权限检查、数据库连接管理等。

特点

- 1 **作用范围**：仅影响当前路径操作（单个 API 端点）。
- 2 **执行顺序**：依赖项在路由处理函数之前执行。
- 3 **参数传递**：依赖项可以访问请求参数（如 **Query**、**Path**、**Header** 等）。
- 4 **返回值**：依赖项可以返回数据，供路由处理函数使用。

```
1 from fastapi import FastAPI, Depends,
   HTTPException, status
2
3 app = FastAPI()
4
5 # 依赖函数：检查用户权限
6 def check_user_permission(token: str =
   Header(...)):
7     if token != "secret-token":
8         raise HTTPException(
9
10        status_code=status.HTTP_403_FORBIDDEN,
11        detail="Invalid token"
12    )
```

```

12         return {"user": "admin"}
13
14 # 路径级依赖注入
15 @app.get("/admin/", dependencies=
    [Depends(check_user_permission)])
16 async def admin_dashboard():
17     return {"message": "Welcome to Admin
    Dashboard"}

```

代码解析

- ① `check_user_permission` 是一个依赖函数，检查请求头中的 `token`
- ② 如果 `token` 无效，抛出 `HTTPException` (403 错误)
- ③ 如果 `token` 有效，返回用户信息 `{"user": "admin"}`
- ④ `admin_dashboard` 路由使用 `dependencies=[Depends(check_user_permission)]` 注入依赖

```

1 from fastapi import FastAPI, Depends, Query
2
3 app = FastAPI()
4
5 # 依赖函数：获取分页参数
6 def get_pagination_params(
7     page: int = Query(1, ge=1),
8     page_size: int = Query(10, ge=1, le=100)
9 ):
10     return {"page": page, "page_size":
    page_size}
11
12 # 路径级依赖注入
13 @app.get("/items/")
14 async def list_items(pagination: dict =
    Depends(get_pagination_params)):
15     return {

```

```
16         "items": ["item1", "item2",  
17         "item3"],  
18         "current_page": pagination["page"],  
19         "page_size": pagination["page_size"]  
20     }
```

代码解析

- 1 `get_pagination_params` 是一个依赖函数，接收 `page` 和 `page_size` 查询参数。
- 2 `page` 必须 ≥ 1 ，`page_size` 必须在 1~100 之间。
- 3 `list_items` 路由通过 `Depends(get_pagination_params)` 获取分页参数

实时效果反馈

1. 路径级依赖注入的作用范围是什么？

- A 整个 FastAPI 应用的所有路由
- B 当前路由（单个 API 端点）
- C 所有 `APIRouter` 下的路由
- D 仅限异步路由

2. 以下哪种情况适合使用路径级依赖注入？

- A 全局日志记录（所有请求都需要）
- B 仅某个特定 API 需要权限检查
- C 数据库连接池管理（所有路由共享）
- D 整个应用的中间件处理

答案

1=>B 2=>B

FastAPI 依赖注入-路由级



在 FastAPI 中，**路由级依赖注入**允许我们为整个 `APIRouter` 下的所有路径操作（API 端点）注入相同的依赖项。这种方式适用于**多个路由共享相同逻辑**的情况，例如：

- **身份验证**（所有路由都需要检查 JWT Token）
- **数据库会话管理**（所有路由都需要访问数据库）
- **日志记录**（所有路由都需要记录请求信息）

特点

- 1 **作用范围**：影响 `APIRouter` 下的所有路由。
- 2 **执行顺序**：依赖项在每个路由处理函数之前执行。
- 3 **代码复用**：避免在每个路由重复编写相同逻辑。
- 4 **可组合性**：可以和路径级依赖一起使用（路径级依赖优先级更高）。

示例代码

```
1 from fastapi import FastAPI, APIRouter, Depends, HTTPException, Header
2
3 app = FastAPI()
4 router = APIRouter(dependencies=[Depends(check_auth)]) # 路由级依赖
```

```

5
6 # 依赖函数：检查用户权限
7 def check_auth(token: str = Header(...)):
8     if token != "secret-token":
9         raise HTTPException(status_code=403,
10 detail="Invalid token")
11     return {"user": "admin"}
12
13 # 路由1：受保护的路由
14 @router.get("/admin/dashboard")
15 async def admin_dashboard():
16     return {"message": "Welcome to Admin
17 Dashboard"}
18
19 # 路由2：受保护的路由
20 @router.get("/admin/users")
21 async def list_users():
22     return {"users": ["user1", "user2"]}
23
24 app.include_router(router, prefix="/api")

```

代码解析

- ① `check_auth` 是一个依赖函数，检查请求头中的 `token`
- ② `router = APIRouter(dependencies=[Depends(check_auth)])` 表示所有该路由下的 API 都会自动执行 `check_auth`
- ③ 访问 `/api/admin/dashboard` 或 `/api/admin/users` 时，都会先检查 `token`

实时效果反馈

1. 路由级依赖注入的作用范围是什么？

- A** 整个 FastAPI 应用的所有路由

- ☐ B 当前 `APIRouter` 下的所有路由
- ☐ C 仅限异步路由
- ☐ D 仅限 GET 请求

2. 以下哪种情况适合使用路由级依赖注入？

- ☐ A 仅某个特定 API 需要权限检查
- ☐ B 全局日志记录（所有请求都需要）
- ☐ C 所有 `/admin` 下的路由都需要身份验证
- ☐ D 仅限 POST 请求的参数验证

答案

1=>B 2=>C

FastAPI 依赖注入_全局级



在 FastAPI 中，**全局级依赖注入**允许我们为**整个应用的所有路由**注入相同的依赖项。这种方式适用于**所有 API 都需要执行的逻辑**

特点

- ① **作用范围**：影响整个 FastAPI 应用的所有路由。
- ② **执行顺序**：依赖项在所有路由处理函数之前执行。
- ③ **适用于**：跨路由的通用逻辑（如日志、认证、限流）。
- ④ **优先级**：低于路径级和路由级依赖（可以被覆盖）。

示例代码

```
1 from fastapi import FastAPI, Depends, Request
2
3 app = FastAPI(dependencies=[Depends(log_request)]) # 全局级依赖
4
5 # 依赖函数：记录请求信息
6 async def log_request(request: Request):
7     print(f"Request received: {request.method} {request.url}")
8     return {"logged": True}
9
10 # 所有路由都会自动记录请求
11 @app.get("/items/")
12 async def list_items():
13     return {"items": ["item1", "item2"]}
14
15 @app.post("/items/")
16 async def create_item():
17     return {"message": "Item created"}
```

代码解析

- ① `log_request` 是一个依赖函数，记录请求的 `method` 和 `url`。
- ② `app = FastAPI(dependencies=[Depends(log_request)])` 使得**所有路由自动记录请求**。
- ③ 无需在每个路由单独写日志逻辑，减少重复代码。

实时效果反馈

1. 全局级依赖注入的作用范围是什么？？

- A 整个 FastAPI 应用的所有路由
- B 当前 `APIRouter` 下的所有路由
- C 仅限异步路由
- D 仅限 GET 请求

2. 以下哪种情况适合使用全局级依赖注入？

- A 仅 `/admin` 路由需要权限检查
- B 所有请求都需要记录访问日志
- C 某个特定 API 需要数据库连接
- D 仅限 GET 请求的参数验证

答案

1=>A 2=>B

FastAPI 依赖注入_嵌套依赖



在 FastAPI 中，**嵌套依赖注入**允许一个依赖项本身依赖于另一个依赖项，形成**依赖链**。这种方式适用于**复杂业务逻辑的拆分和复用**，例如：

- **权限验证 + 用户信息获取**（先验证 Token，再查询用户信息）
- **数据库连接 + 事务管理**（先获取数据库连接，再开启事务）
- **参数校验 + 业务逻辑处理**（先校验参数，再执行业务逻辑）

特点

- ① **依赖链**：一个依赖可以调用另一个依赖。
- ② **执行顺序**：从最外层依赖开始，逐层向内执行。
- ③ **代码复用**：避免重复逻辑，提高可维护性。
- ④ **支持异步**：嵌套依赖可以是同步或异步函数。

示例代码

```
1 from fastapi import FastAPI, Depends, HTTPException, Header
2
3 app = FastAPI()
4
5 # 依赖1: 验证 Token
6 def verify_token(token: str = Header(...)):
7     if token != "secret-token":
```

```

8         raise HTTPException(status_code=403,
detail="Invalid token")
9     return token
10
11 # 依赖2: 获取用户信息 (依赖于 verify_token)
12 def get_current_user(token: str =
Depends(verify_token)):
13     return {"user": "admin", "token": token}
14
15 # 使用嵌套依赖
16 @app.get("/user/")
17 async def get_user_info(user: dict =
Depends(get_current_user)):
18     return user

```

代码解析

- ① `verify_token` 验证请求头中的 `token`。
- ② `get_current_user` 依赖于 `verify_token`，并返回用户信息。
- ③ `get_user_info` 使用 `get_current_user`，形成依赖链：先执行 `verify_token` → 再执行 `get_current_user` → 最后执行 `get_user_info`。

实时效果反馈

1. 嵌套依赖的执行顺序是什么？

- A** 从最内层依赖开始，逐层向外执行
- B** 从最外层依赖开始，逐层向内执行
- C** 随机顺序执行
- D** 同时执行所有依赖

答案

1=>B

FastAPI 依赖注入_类依赖



在 FastAPI 中，**类依赖注入**允许我们使用**类 (Class)** 作为依赖项，而不仅仅是函数。这种方式特别适合**封装复杂逻辑**或**需要维护状态**的场景，例如：

- **数据库服务**（封装 CRUD 操作）
- **外部 API 客户端**（封装 HTTP 请求）
- **配置管理**（读取和缓存配置）

特点

- 1 **面向对象**：可以封装属性和方法。
- 2 **可复用性**：类实例可以在多个路由中共享。
- 3 **依赖嵌套**：类可以依赖其他类或函数。
- 4 **支持初始化参数**：可以通过 `__init__` 传递配置。

示例代码

```
1 from fastapi import FastAPI, Depends
2
3 app = FastAPI()
4
5 # 类依赖: 用户服务
6 class UserService:
7     def __init__(self, db_connection: str =
8         "default"):
9         self.db = db_connection # 模拟数据库
10        连接
11
12    def get_user(self, user_id: int):
13        return {"id": user_id, "name":
14            "Alice", "db": self.db}
15
16 # 依赖函数: 创建 UserService 实例
17 def get_user_service():
18     return UserService(db_connection="prod-
19         db")
20
21 # 使用类依赖
22 @app.get("/users/{user_id}")
23 async def read_user(
24     user_id: int,
25     user_service: UserService =
26     Depends(get_user_service)
27 ):
28     return user_service.get_user(user_id)
```

代码解析

- 1 `UserService` 是一个类依赖，封装了用户相关操作。
- 2 `get_user_service` 返回 `UserService` 实例（可在这里初始化配置）。
- 3 `read_user` 通过 `Depends(get_user_service)` 注入 `UserService` 实例。

实时效果反馈

1. 类依赖的主要优势是什么？

- A 只能用于同步代码
- B 可以封装状态和方法
- C 必须全局单例
- D 不能嵌套其他依赖

答案

1=>B

领域驱动设计（DDD）



DDD (Domain-Driven Design) 是一个让程序员和产品经理（或者任何提需求的人）不再“鸡同鸭讲”的超级沟通和做事的办法。

以后工作时，可能碰到的问题：

- 代码越写越乱，改一个功能要动十几个地方。
- 数据库表设计好了，但业务逻辑全散在 Service 里，没人看得懂。
- 产品经理说“会员下单有折扣”，你却不知道该写在哪。
- 和产品沟通时总对不上口径：“你说的‘下单成功’到底指啥？”

这些问题，DDD 就是来解决的

1. DDD 的核心思想

- ① **业务优先**：先搞懂业务是怎么运作的，而不是一上来就建表。
- ② **统一语言**：程序员、产品经理、业务方用**同一个词**描述同一个东西。
- ③ **边界清晰**：把大系统拆成几个“小王国”，每个王国自己管自己

2. DDD 核心概念

- ① **领域**：软件要解决的那个业务范围
- ② **通用语言**：团队里所有人（程序员、产品经理、测试等）约定好的一套“黑话”。讨论业务、写代码、写文档都用这套话，不准出现第二种说法

关键点：通用语言是DDD的基石，是防止“鸡同鸭讲”的终极武器。它的词直接变成你的类名、方法名

- 3 **限界上下文：给“通用语言”划边界。** 同一个词，在不同的部门（上下文）里，意思可能完全不同。限界上下文就是把这些部门隔开，让每个部门内部用自己的“方言”而不会混乱。

例子：“订单”这个词：

- 在【用户下单上下文】里，“订单”关心的是：菜品、价格、优惠券、送餐地址。
- 在【后厨制作上下文】里，“订单”关心的是：做什么菜、做菜顺序、出餐口。
- 在【配送调度上下文】里，“订单”关心的是：取餐地点、送餐地点、路径规划。

关键点：你不能把这三个上下文的“订单”混在一起设计成一个巨无霸 **Order** 类，会复杂到爆炸。**限界上下文就是告诉你，应该把它们当成三个不同的东西来开发，甚至可以做成三个不同的微服务。这是降低复杂度的超级大招。**

- 4 **实体：有唯一ID的东西，你看重的是“它是谁”，而不是它“是什么样”。** 它会变化，但它的身份（ID）不变

例子：用户是一个实体。因为每个用户都有一个唯一的用户ID（比如123）。用户“张三”今天可以改名叫“张四”，密码也可以改，但他的用户ID 123没变，我们就知道他还是那个“张三”。

- 5 **值对象：没ID，只看属性值的东西。** 你看重的是“它是什么样”，如果两个东西的所有属性值一样，就可以认为它们是同一个东西

例子：地址是一个典型的值对象。你有一个地址对象：**国家=中国，城市=北京，街道=xxx路1号**。如果另一个地址对象的这些属性值完全一样，那它们就是相等的，即使它们在内存里是两个不同的对象。值对象通常还是**不可变的**，你的地址信息一旦创建就不会轻易改动，要改就是整个换掉。

- 6 **聚合根：一组相关对象的“老大”，外部只能通过它来访问这组对象，它是保证业务一致性的边界。**

关键点：

- 聚合根是实体（有ID），但不是所有实体都是聚合根。
- 一个聚合包含多个实体和值对象，但外部只能通过聚合根操作它们（类似“黑盒”）。

- 聚合根负责维护聚合内部的业务规则一致性。

为什么需要聚合根？

假设“订单”是一个聚合根，包含：

- 订单实体（根）
- 订单项（子实体）
- 配送地址（值对象）

没有聚合根时的混乱：

- 代码可能直接修改 `OrderItem` 的价格，导致订单总价不一致。
- 可能误删 `Address`，导致订单数据不完整。

有聚合根后的规则：

- 要改订单项？必须通过 `Order` 聚合根的方法（如 `order.updateItem()`），它会自动重新计算总价。
- 要删地址？必须调用 `Order.changeAddress()`，确保新地址有效后才替换旧地址。

如何识别聚合根？

- ① **业务高频操作点**：比如“下单”场景中，`Order` 是天然聚合根。
 - ② **强一致性需求**：比如“银行账户”转账，`Account` 必须是聚合根，保证余额变化原子性。
 - ③ **生命周期控制**：比如“论坛帖子”删除时，连带回复一起删，`Post` 就是聚合根
- ⑦ **领域服务**：处理那些不属于任何实体/值对象的业务逻辑，通常是跨聚合的、无状态的、需要协调多个领域对象的操作

关键点：

- 领域服务是**领域层**的组件（不是应用服务！）。
- 它本身**没有状态**（不保存数据），但操作有状态的领域对象。
- 命名通常是动词短语（如 `TransferService`），而实体/值对象是名词（如 `Account`）。

假设你要实现“从账户A转账100元到账户B”。

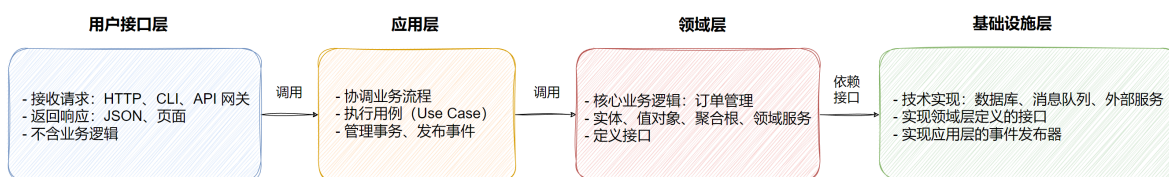
这个操作涉及两个账户，你不能把 `transferTo` 方法放在 `Account` 类里，因为那会变成：

```
1 accountA.transferTo(accountB, 100);
```

解决方案: `TransferService.execute(from_account, to_account, amount)`

3. DDD 分层架构

领域驱动设计（DDD）推荐的分层架构，各层职责明确、依赖单向，确保业务核心（领域层）不受技术细节污染。



① 用户接口层（User Interface Layer）

- **别名**：表现层、Web 层、接口层、Controller 层

- **职责**：

- 接收外部请求（如 HTTP、RPC、CLI）
- 解析输入参数（如 JSON、表单）
- 执行基础校验（如非空、格式）
- 调用应用层服务完成业务操作
- 返回响应结果（如 JSON、页面）

- **不包含**：

- 业务规则判断
- 领域逻辑

- **依赖**：

- 仅依赖 **应用层**（调用应用服务）
- 示例：`OrderController` → `PlaceOrderService`

② 应用层（Application Layer）

- **别名**：用例层、服务门面层

- **职责**：

- 协调领域对象完成一个完整的业务用例（如“下单”、“转账”）
- 处理事务边界（如开启事务）
- 发布领域事件

- 转换 DTO（数据传输对象）
- 不包含核心业务规则（只“指挥”，不“决策”）

- **组件示例：**

- 应用服务（ **PlaceOrderService** ）
- Command/Query 对象
- DTO（Data Transfer Object）

- **依赖：**

- 依赖 **领域层**（使用实体、聚合、领域服务）
- 依赖 **基础设施层**（获取 Repository 实现）
- 被 **用户界面层** 调用

3 领域层（Domain Layer）

- **别名：**模型层、核心层

- **职责：**

- 包含系统的核心业务逻辑和规则
- 定义领域模型：
 - 实体
 - 值对象
 - 聚合根
 - 领域服务
 - 领域事件
- 保证业务一致性（如订单金额不能为负）
- **不依赖任何外部框架**（如 Flask、Django、FastAPI、SQLAlchemy）
- **关键原则：**
 - 业务逻辑必须在这里实现
 - 聚合根负责维护内部一致性
- **依赖：**
 - **无依赖**（理想情况下不依赖其他层）
 - 被 **应用层** 调用

4 基础设施层（Infrastructure Layer）

- **别名：**数据访问层、技术实现层

- **职责：**
 - 提供技术实现，支撑上层运行
 - 实现领域层定义的接口，例如：
 - `OrderRepository` 接口的具体实现（如 JPA、MyBatis）
 - 消息队列发送器（如 KafkaProducer）
 - 外部 API 调用（如支付网关）
 - 处理数据库、缓存、文件、邮件等底层操作
- **组件示例：**
 - Repository 实现类
 - 事件发布者
 - 第三方客户端
- **依赖：**
 - 依赖 **领域层**（实现其接口）
 - 依赖 **应用层**（如监听应用事件）
 - 被 **应用层** 和 **领域事件处理器** 调用

重点：业务逻辑只在领域层，其他层只是“搬运工”

4. DDD 适合什么时候用？

场景	是否推荐 DDD
简单的 CRUD 系统（如后台管理）	✗ 不推荐，太重
电商、金融、ERP 等复杂业务	✓ 强烈推荐
团队协作多，沟通成本高	✓ 推荐（通用语言有用）
快速验证的 MVP 项目	✗ 先快速做，再重构

基于DDD 开发FastAPI项目



🚀 开发流程概览



🎯 核心功能

- ☒ 用户注册/登录
- ☒ 订单查询
- ☒ DDD分层架构

项目目录结构

```

1 fastapi_code/
2 |— main.py # 应用入口文件
3 |— pyproject.toml # 项目配置和依赖管理
4 |— README.md # 项目说明文档
5 |— uv.lock # 依赖锁定文件
6 |— config/ # 配置模块
7 |   |— settings.py # 应用配置管理
  
```

```

8  |— data/                                # 数据目
   录
9  |   |— db.sqlite3                      #
   SQLite数据库文件
10 |   |— db.sqlite3-shm                  #
   SQLite共享内存文件
11 |   └─ db.sqlite3-wal                  #
   SQLite预写日志文件
12 └─ app/                                # 主应用
   目录
13   |— __init__.py                       # 应用包
   初始化
14   |— domain/                           # 领域层
15   |   |— __init__.py
16   |   |— shared/                       # 共享领
   域对象
17   |   |   |— __init__.py
18   |   |   └─ vo.py                     # 值对象
   定义
19   |   └─ user/                          # 用户领
   域
20   |       |— __init__.py
21   |       |— entity.py                 # 用户实
   体
22   |       └─ repository.py             # 用户仓
   储接口
23   |— application/                      # 应用层
24   |   |— __init__.py
25   |   |— common/                      # 通用应
   用组件
26   |   |   |— __init__.py

```

```

27 | | └─ exception.py # 业务异
    常定义
28 | └─ user/ # 用户应
    用服务
29 | └─ __init__.py
30 | └─ commands/ # 命令处
    理器
31 | | └─ __init__.py
32 | | └─ login_user.py # 用户登
    录命令
33 | | └─ register_user.py # 用户注
    册命令
34 | └─ queries/ # 查询处
    理器
35 | └─ __init__.py
36 | └─ get_orders.py # 获取订
    单查询
37 └─ infrastructure/ # 基础设
    施层
38 | └─ __init__.py
39 | └─ database/ # 数据库
    相关
40 | | └─ __init__.py
41 | | └─ orm_models.py # ORM模
    型定义
42 | | └─ mappers.py # 实体映
    射器
43 | └─ repository/ # 仓储实
    现
44 | └─ __init__.py
45 | └─ user_impl.py # 用户仓
    储实现

```

```

46     └─ interface/                                # 接口层
47         └─ __init__.py
48         └─ dependency.py                          # 依赖注
           入配置
49     └─ api/                                        # API接
           口
50         └─ __init__.py
51         └─ v1/                                    # API版
           本1
52             └─ __init__.py
53             └─ auth_router.py                    # 认证路
           由
54             └─ order_router.py                  # 订单路
           由

```

六个开发阶段

第一阶段：项目基础设置 (2个文件)

目标: 建立项目基础环境和配置

- ① 项目初始化 - `pyproject.toml` - 定义依赖和项目信息
- ② 应用配置 - `config/settings.py` - 统一配置管理

第二阶段：领域层开发 (3个文件)

目标: 定义核心业务模型和规则

- ① 共享值对象 - `app/domain/shared/vo.py` - 定义UserId等值对象
- ② 用户实体 - `app/domain/user/entity.py` - 定义User业务实体
- ③ 仓储接口 - `app/domain/user/repository.py` - 定义数据访问抽象

第三阶段：基础设施层开发 (3个文件)

目标: 实现数据持久化和外部依赖

- ① ORM模型 - `app/infrastructure/database/orm_models.py` - 数据库表结构
- ② 实体映射器 - `app/infrastructure/database/mappers.py` - 实体与ORM转换

- ③ 仓储实现 - `app/infrastructure/repository/user_impl.py` - 仓储接口实现

第四阶段：应用层开发 (4个文件)

目标: 实现业务逻辑和用例

- ① 通用异常 - `app/application/common/exception.py` - 业务异常定义
- ② 用户注册命令 - `app/application/user/commands/register_user.py` - 注册业务逻辑
- ③ 用户登录命令 - `app/application/user/commands/login_user.py` - 登录业务逻辑
- ④ 订单查询 - `app/application/user/queries/get_orders.py` - 查询业务逻辑

第五阶段：接口层开发 (3个文件)

目标: 提供API接口和依赖注入

- ① 依赖注入 - `app/interface/dependency.py` - 依赖管理配置
- ② 认证路由 - `app/interface/api/v1/auth_router.py` - 认证API接口
- ③ 订单路由 - `app/interface/api/v1/order_router.py` - 订单API接口

第六阶段：应用入口 (1个文件)

目标: 启动应用和生命周期管理

- ① 主应用文件 - `main.py` - 应用启动和配置

详细开发内容

第一阶段：项目基础设置

目标: 建立项目基础环境和配置

1. 项目初始化

- **文件:** `pyproject.toml`
- **目的:** 定义项目依赖和配置
- **具体开发内容:**
 - 设置项目基本信息: `name = "fastapi-code2", version = "0.1.0"`
 - 添加核心依赖:

- fastapi>=0.104.1 (Web框架)
- uvicorn[standard]>=0.24.0 (ASGI服务器)
- tortoise-orm[asyncpg]>=0.20.0 (异步ORM)
- python-dotenv>=1.0.0 (环境变量管理)
- pydantic>=2.5.0 (数据验证)
- pydantic-settings>=2.1.0 (配置管理)

2. 应用配置

- 文件: `config/settings.py`
- 目的: 统一管理应用配置
- 具体开发内容:
 - 创建Settings类继承BaseSettings
 - 定义应用配置: app_name、app_version、debug、secret_key
 - 定义数据库配置: db_url = "sqlite:///data/db.sqlite3"
 - 配置环境变量支持: env_file = ".env"
 - 创建全局配置实例: settings = Settings()

第二阶段：领域层开发

目标: 定义核心业务模型和规则

3. 共享值对象

- 文件: `app/domain/shared/vo.py`
- 目的: 定义领域中的值对象
- 具体开发内容:
 - 创建UserId值对象类:

```

1 @dataclass(frozen=True)
2 class UserId:
3     value: int
4     def __post_init__(self):
5         if self.value <= 0:
6             raise ValueError("用户ID必须大
于0")

```

- 使用dataclass和frozen=True确保不可变性

- 添加业务规则验证：用户ID必须大于0

4. 用户实体

- 文件: `app/domain/user/entity.py`

- 目的: 定义用户领域实体

- 具体开发内容:

- 创建User实体类:

```
1 @dataclass
2 class User:
3     id: Optional[UserId]
4     username: str
5     password: str
```

- 实现业务方法:

- `verify_password(password: str) -> bool`: 验证密码
- `change_password(new_password: str) -> None`: 修改密码

- 添加数据验证逻辑：用户名和密码不能为空

- 导入UserId值对象: `from ..shared.vo import UserId`

```
1 """
2 用户实体
3 """
4 from dataclasses import dataclass
5 from typing import Optional
6 from ..shared.vo import UserId
7
8
9 @dataclass
10 class User:
11     """用户实体"""
12     id: Optional[UserId]
13     username: str
14     password: str # 明文密码，按需求不加密
```

```

15
16     def __post_init__(self):
17         if not self.username:
18             raise ValueError("用户名不能
为空")
19         if not self.password:
20             raise ValueError("密码不能为
空")
21
22     def verify_password(self, password:
str) -> bool:
23         """验证密码"""
24         return self.password ==
password
25
26     def change_password(self,
new_password: str) -> None:
27         """修改密码"""
28         if not new_password:
29             raise ValueError("新密码不能
为空")
30         self.password = new_password

```

5. 仓储接口

- **文件:** `app/domain/user/repository.py`
- **目的:** 定义用户仓储抽象接口
- **具体开发内容:**
 - 创建UserRepository抽象基类
 - 导入必要的类型: User、UserId、Optional、List
 - 遵循依赖倒置原则, 定义抽象接口

```

1     """

```

```
2  用户仓储接口
3  """
4  from abc import ABC, abstractmethod
5  from typing import Optional, List
6  from .entity import User
7  from ..shared.vo import UserId
8
9  class UserRepository(ABC):
10     """用户仓储接口"""
11     @abstractmethod
12     async def save(self, user: User) ->
User:
13         """保存用户"""
14         pass
15     @abstractmethod
16     async def find_by_id(self, user_id:
UserId) -> Optional[User]:
17         """根据ID查找用户"""
18         pass
19     @abstractmethod
20     async def find_by_username(self,
username: str) -> Optional[User]:
21         """根据用户名查找用户"""
22         pass
23     @abstractmethod
24     async def exists_by_username(self,
username: str) -> bool:
25         """检查用户名是否存在"""
26         pass
27     @abstractmethod
28     async def delete(self, user_id:
UserId) -> bool:
```

```

29         """删除用户"""
30         pass
31     @abstractmethod
32     async def find_all(self) ->
List[User]:
33         """查找所有用户"""
34         pass

```

第三阶段：基础设施层开发

目标: 实现数据持久化和外部依赖

6. ORM模型

- **文件:** `app/infrastructure/database/orm_models.py`
- **目的:** 定义数据库表结构
- **具体开发内容:**
 - 创建UserORM模型类:

```

1  """
2  Tortoise ORM 模型
3  """
4  from tortoise.models import Model
5  from tortoise import fields
6
7
8  class UserORM(Model):
9      """用户ORM模型"""
10     id = fields.IntField(pk=True)
11     username =
fields.CharField(max_length=50,
unique=True)

```

```

12     password =
fields.CharField(max_length=255) # 明文
    密码
13     created_at =
fields.DateTimeField(auto_now_add=True)
14     updated_at =
fields.DateTimeField(auto_now=True)
15
16     class Meta:
17         table = "users"

```

- 导入Tortoise ORM: from tortoise.models import Model, from tortoise import fields
- 设置字段约束: username唯一性、密码长度限制
- 配置自动时间戳: created_at、updated_at

7. 实体映射器

- 文件: `app/infrastructure/database/mappers.py`
- 目的: 实现领域实体与ORM模型的转换
- 具体开发内容:
 - 创建UserMapper类:

```

1     """
2     实体与ORM模型映射器
3     """
4     from typing import Optional
5     from app.domain.user.entity import User
6     from app.domain.shared.vo import UserId
7     from
app.infrastructure.database.orm_models
    import UserORM
8
9

```

```

10 class UserMapper:
11     """用户实体映射器"""
12
13     @staticmethod
14     def to_entity(orm_model: UserORM) -
15 > User:
16         """ORM模型转实体"""
17         return User(
18             id=UserId(orm_model.id) if
19 orm_model.id else None,
20
21             username=orm_model.username,
22             password=orm_model.password
23         )
24
25     @staticmethod
26     def to_orm(entity: User) ->
27 UserORM:
28         """实体转ORM模型"""
29         orm_model = UserORM()
30         if entity.id:
31             orm_model.id =
32 entity.id.value
33         orm_model.username =
34 entity.username
35         orm_model.password =
36 entity.password
37         return orm_model

```

- 导入相关类型：User、UserId、UserORM
- 处理ID值对象的转换：UserId <-> int
- 实现双向转换：实体转ORM、ORM转实体

8. 仓储实现

- 文件: `app/infrastructure/repository/user_impl.py`
- 目的: 实现用户仓储接口
- 具体开发内容:
 - 创建TortoiseUserRepository类:

```
1  """
2  用户仓储具体实现
3  """
4  from typing import Optional, List
5  from tortoise.exceptions import
   IntegrityError
6  from app.domain.user.entity import User
7  from app.domain.user.repository import
   UserRepository
8  from app.domain.shared.vo import UserId
9  from
   app.infrastructure.database.orm_models
   import UserORM
10 from
   app.infrastructure.database.mappers
   import UserMapper
11
12
13 class
   TortoiseUserRepository(UserRepository):
14     """基于Tortoise ORM的用户仓储实现"""
15
16     async def save(self, user: User) ->
   User:
17         """保存用户"""
18         try:
```

```
19         orm_model =
UserMapper.to_orm(user)
20         await orm_model.save()
21         return
UserMapper.to_entity(orm_model)
22         except IntegrityError as e:
23             if "UNIQUE constraint
failed" in str(e):
24                 raise ValueError(f"用户
名 '{user.username}' 已存在")
25                 raise e
26
27     async def find_by_id(self, user_id:
UserId) -> Optional[User]:
28         """根据ID查找用户"""
29         orm_model = await
UserORM.get_or_none(id=user_id.value)
30         if orm_model:
31             return
UserMapper.to_entity(orm_model)
32         return None
33
34     async def find_by_username(self,
username: str) -> Optional[User]:
35         """根据用户名查找用户"""
36         orm_model = await
UserORM.get_or_none(username=username)
37         if orm_model:
38             return
UserMapper.to_entity(orm_model)
39         return None
40
```

```

41     async def exists_by_username(self,
username: str) -> bool:
42         """检查用户名是否存在"""
43         return await
UserORM.filter(username=username).exists()
44
45     async def delete(self, user_id:
UserId) -> bool:
46         """删除用户"""
47         deleted_count = await
UserORM.filter(id=user_id.value).delete()
48         return deleted_count > 0
49
50     async def find_all(self) ->
List[User]:
51         """查找所有用户"""
52         orm_models = await
UserORM.all()
53         return
[UserMapper.to_entity(orm_model) for
orm_model in orm_models]

```

- 导入必要的依赖：UserRepository、User、UserId、UserORM、UserMapper
- 实现所有仓储接口方法
- 处理数据库异常：IntegrityError转换为业务异常
- 使用UserMapper进行实体转换

第四阶段：应用层开发

目标: 实现业务逻辑和用例

9. 通用异常

- 文件: `app/application/common/exception.py`
- 目的: 定义应用层通用异常
- 具体开发内容:
 - 创建DomainException基类:

```
1  """
2  通用业务异常
3  """
4  from typing import Optional
5
6
7  class DomainException(Exception):
8      """领域异常基类"""
9      def __init__(self, message: str,
10 code: Optional[str] = None):
11         self.message = message
12         self.code = code
13         super().__init__(self.message)
14
15  class AuthError(DomainException):
16      """认证异常"""
17      def __init__(self, message: str =
18 "认证失败"):
19         super().__init__(message,
20 "AUTH_ERROR")
```

```
21 class DuplicateError(DomainException):
22     """重复数据异常"""
23     def __init__(self, message: str =
24         "数据已存在"):
25         super().__init__(message,
26             "DUPLICATE_ERROR")
27
28 class NotFoundError(DomainException):
29     """未找到异常"""
30     def __init__(self, message: str =
31         "数据未找到"):
32         super().__init__(message,
33             "NOT_FOUND_ERROR")
34
35 class ValidationError(DomainException):
36     """验证异常"""
37     def __init__(self, message: str =
38         "数据验证失败"):
39         super().__init__(message,
40             "VALIDATION_ERROR")
```

- 定义具体业务异常：
 - AuthError: 认证失败异常
 - DuplicateError: 重复数据异常
 - NotFoundError: 未找到异常
 - ValidationError: 验证异常
- 统一异常格式: message + code
- 导入Optional类型支持

10. 用户注册命令

- 文件: `app/application/user/commands/register_user.py`
- 目的: 处理用户注册业务逻辑
- 具体开发内容:
 - 定义命令和结果类:

```
1  """
2  用户注册命令处理器
3  """
4  from dataclasses import dataclass
5  from app.domain.user.entity import User
6  from app.domain.user.repository import
   UserRepository
7  from app.application.common.exception
   import DuplicateError, ValidationError
8
9  @dataclass
10 class RegisterUserCommand:
11     """用户注册命令"""
12     username: str
13     password: str
14
15 @dataclass
16 class RegisterUserResult:
17     """用户注册结果"""
18     user_id: int
19     username: str
20
21 class RegisterUserHandler:
22     """用户注册命令处理器"""
23
24     def __init__(self, user_repository:
   UserRepository):
```

```
23         self.user_repository =  
user_repository  
24  
25     async def handle(self, command:  
RegisterUserCommand) ->  
RegisterUserResult:  
26         """处理用户注册命令"""  
27         # 验证输入  
28         if not command.username or not  
command.username.strip():  
29             raise ValidationError("用户  
名不能为空")  
30  
31         if not command.password or not  
command.password.strip():  
32             raise ValidationError("密码  
不能为空")  
33  
34         # 检查用户名是否已存在  
35         if await  
self.user_repository.exists_by_username  
(command.username):  
36             raise DuplicateError(f"用户  
名 '{command.username}' 已存在")  
37  
38         # 创建用户实体  
39         user = User(  
40             id=None, # 新用户, ID由数据库  
生成  
41  
username=command.username.strip(),
```

```

42     password=command.password.strip()
43     )
44
45     # 保存用户
46     saved_user = await
self.user_repository.save(user)
47
48     return RegisterUserResult(
49         user_id=saved_user.id.value
if saved_user.id else 0,
50
51     username=saved_user.username
52     )

```

- 实现RegisterUserHandler类：
 - 构造函数注入UserRepository
 - handle方法处理注册逻辑
 - 输入验证：用户名和密码不能为空
 - 业务规则检查：用户名唯一性
 - 创建User实体并保存
- 导入相关依赖：User、UserRepository、UserId、异常类

11. 用户登录命令

- 文件: `app/application/user/commands/login_user.py`
- 目的: 处理用户登录业务逻辑
- 具体开发内容:
 - 定义命令和结果类:

```

1  """
2  用户登录命令处理器
3  """
4  from dataclasses import dataclass

```

```
5 from app.domain.user.repository import
  UserRepository
6 from app.application.common.exception
  import AuthError, ValidationError
7
8
9 @dataclass
10 class LoginUserCommand:
11     """用户登录命令"""
12     username: str
13     password: str
14
15
16 @dataclass
17 class LoginUserResult:
18     """用户登录结果"""
19     user_id: int
20     username: str
21
22
23 class LoginUserHandler:
24     """用户登录命令处理器"""
25
26     def __init__(self, user_repository:
  UserRepository):
27         self.user_repository =
  user_repository
28
29     async def handle(self, command:
  LoginUserCommand) -> LoginUserResult:
30         """处理用户登录命令"""
31         # 验证输入
```

```

32         if not command.username or not
command.username.strip():
33             raise ValidationError("用户
名不能为空")
34
35         if not command.password or not
command.password.strip():
36             raise ValidationError("密码
不能为空")
37
38         # 查找用户
39         user = await
self.user_repository.find_by_username(c
ommand.username.strip())
40         if not user:
41             raise AuthError("用户名或密码
错误")
42
43         # 验证密码
44         if not
user.verify_password(command.password.s
trip()):
45             raise AuthError("用户名或密码
错误")
46
47         return LoginUserResult(
48             user_id=user.id.value if
user.id else 0,
49             username=user.username
50         )

```

- 实现LoginUserHandler类:

- 构造函数注入UserRepository

- handle方法处理登录逻辑
- 输入验证：用户名和密码不能为空
- 用户查找：根据用户名查找用户
- 密码验证：调用实体验证方法
- 返回登录结果
- 导入相关依赖：User、UserRepository、UserId、异常类

12. 订单查询

- **文件:** `app/application/user/queries/get_orders.py`
- **目的:** 处理订单查询业务逻辑
- **具体开发内容:**
 - 定义查询和结果类:

```

1  """
2  获取订单查询处理器
3  """
4  from dataclasses import dataclass
5  from typing import List, Optional
6  from app.application.common.exception
   import ValidationError
7
8
9  @dataclass
10 class GetOrdersQuery:
11     """获取订单查询"""
12     user_id: int
13     limit: Optional[int] = 10
14     offset: Optional[int] = 0
15
16
17 @dataclass
18 class OrderDto:
19     """订单数据传输对象"""

```

```
20     id: int
21     order_number: str
22     total_amount: float
23     status: str
24     created_at: str
25     updated_at: str
26
27
28 @dataclass
29 class GetOrdersResult:
30     """获取订单结果"""
31     orders: List[OrderDto]
32     total: int
33
34
35 class GetOrdersHandler:
36     """获取订单查询处理器"""
37
38     def __init__(self):
39         # 这里应该注入订单仓储，但为了简化，
40         # 我们使用模拟数据
41         pass
42
43     async def handle(self, query:
44         GetOrdersQuery) -> GetOrdersResult:
45         """处理获取订单查询"""
46         # 验证输入
47         if query.user_id <= 0:
48             raise ValidationError("用户
49             ID必须大于0")
```

```
48         if query.limit and query.limit
49             <= 0:
50                 raise ValidationError("限制
51                 数量必须大于0")
52
53         if query.offset and
54             query.offset < 0:
55             raise ValidationError("偏移
56             量不能为负数")
57
58         # 模拟订单数据（实际应该从仓储获取）
59         mock_orders = [
60             OrderDto(
61                 id=1,
62                 order_number="ORD001",
63                 total_amount=99.99,
64                 status="已完成",
65                 created_at="2030-01-
66                 01T10:00:00",
67                 updated_at="2030-01-
68                 01T12:00:00"
69             ),
70             OrderDto(
71                 id=2,
72                 order_number="ORD002",
73                 total_amount=199.99,
74                 status="处理中",
75                 created_at="2030-01-
76                 02T10:00:00",
77                 updated_at="2030-01-
78                 02T10:00:00"
79             )
80         ]
```

```

72         ]
73
74         # 应用分页
75         start = query.offset or 0
76         end = start + (query.limit or
77             10)
78
79         paginated_orders =
80         mock_orders[start:end]
81
82         return GetOrdersResult(
83             orders=paginated_orders,
84             total=len(mock_orders)
85         )

```

- 实现GetOrdersHandler类：
 - handle方法处理查询逻辑
 - 输入验证：user_id、limit、offset验证
 - 模拟订单数据（实际应连接订单仓储）
 - 分页处理：应用limit和offset
 - 返回查询结果
- 导入相关依赖：ValidationError、List、Optional

第五阶段：接口层开发

目标: 提供API接口和依赖注入

13. 依赖注入

- **文件:** `app/interface/dependency.py`
- **目的:** 配置依赖注入容器
- **具体开发内容:**
 - 定义依赖工厂函数:

```

1     """
2     依赖注入容器

```

```
3  """
4  from fastapi import Depends
5  from
   app.infrastructure.repository.user_impl
   import TortoiseUserRepository
6  from app.domain.user.repository import
   UserRepository
7  from
   app.application.user.commands.register_
   user import RegisterUserHandler
8  from
   app.application.user.commands.login_use
   r import LoginUserHandler
9  from
   app.application.user.queries.get_orders
   import GetOrdersHandler
10
11
12 def get_user_repository() ->
   UserRepository:
13     """获取用户仓储实例"""
14     return TortoiseUserRepository()
15 def get_register_user_handler(
16     user_repository: UserRepository =
   Depends(get_user_repository)
17 ) -> RegisterUserHandler:
18     """获取用户注册处理器"""
19     return
   RegisterUserHandler(user_repository)
20 def get_login_user_handler(
21     user_repository: UserRepository =
   Depends(get_user_repository)
```

```

22 ) -> LoginUserHandler:
23     """获取用户登录处理器"""
24     return
25     LoginUserHandler(user_repository)
26 def get_orders_handler() ->
27     GetOrdersHandler:
28     """获取订单查询处理器"""
29     return GetOrdersHandler()

```

- 导入相关依赖：FastAPI Depends、各Handler类、Repository实现
- 使用FastAPI的Depends进行依赖注入
- 连接各层之间的依赖关系

14. 认证路由

- **文件:** `app/interface/api/v1/auth_router.py`
- **目的:** 提供用户认证相关的API接口
- **具体开发内容:**
 - 创建APIRouter实例: `router = APIRouter(prefix="/auth", tags=["认证"])`
 - 定义请求/响应模型:

```

1  """
2  认证和用户路由
3  """
4  from fastapi import APIRouter, Depends,
5  HTTPException, status
6  from pydantic import BaseModel
7  from app.interface.dependency import
8  get_register_user_handler,
9  get_login_user_handler

```

```
7  from
    app.application.user.commands.register_
    user import RegisterUserCommand,
    RegisterUserHandler
8  from
    app.application.user.commands.login_use
    r import LoginUserCommand,
    LoginUserHandler
9  from app.application.common.exception
    import DomainException
10
11
12  router = APIRouter(prefix="/auth",
    tags=["认证"])
13
14
15  class RegisterRequest(BaseModel):
16      """用户注册请求"""
17      username: str
18      password: str
19
20
21  class RegisterResponse(BaseModel):
22      """用户注册响应"""
23      user_id: int
24      username: str
25      message: str
26
27
28  class LoginRequest(BaseModel):
29      """用户登录请求"""
30      username: str
```

```
31     password: str
32
33
34 class LoginResponse(BaseModel):
35     """用户登录响应"""
36     user_id: int
37     username: str
38     message: str
39
40
41 @router.post("/register",
42             response_model=RegisterResponse,
43             status_code=status.HTTP_201_CREATED)
44 async def register_user(
45     request: RegisterRequest,
46     handler: RegisterUserHandler =
47     Depends(get_register_user_handler)
48 ):
49     """用户注册"""
50     try:
51         command = RegisterUserCommand(
52             username=request.username,
53             password=request.password
54         )
55         result = await
56         handler.handle(command)
57
58         return RegisterResponse(
59             user_id=result.user_id,
60             username=result.username,
61             message="用户注册成功"
62         )
```

```
59     except DomainException as e:
60         raise HTTPException(
61
62             status_code=status.HTTP_400_BAD_REQUEST,
63             detail=e.message
64         )
65     except Exception as e:
66         raise HTTPException(
67
68             status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
69             detail="服务器内部错误"
70         )
71
72 @router.post("/login",
73             response_model=LoginResponse)
74 async def login_user(
75     request: LoginRequest,
76     handler: LoginUserHandler =
77     Depends(get_login_user_handler)
78 ):
79     """用户登录"""
80     try:
81         command = LoginUserCommand(
82             username=request.username,
83             password=request.password
84         )
85         result = await
86         handler.handle(command)
87     
```

```
84         return LoginResponse(  
85             user_id=result.user_id,  
86             username=result.username,  
87             message="登录成功"  
88         )  
89     except DomainException as e:  
90         raise HTTPException(  
91  
92             status_code=status.HTTP_401_UNAUTHORIZED,  
93             detail=e.message  
94         )  
95     except Exception as e:  
96         raise HTTPException(  
97  
98             status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,  
99             detail="服务器内部错误"  
100         )
```

- 实现API端点：
 - POST /register: 用户注册
 - POST /login: 用户登录
- 异常处理: DomainException转换为HTTPException
- 状态码映射: 201创建、401未授权、400错误请求

15. 订单路由

- 文件: `app/interface/api/v1/order_router.py`
- 目的: 提供订单相关的API接口
- 具体开发内容:

- 创建APIRouter实例: router = APIRouter(prefix="/orders", tags=["订单"])
- 定义响应模型:

```
1  """
2  订单路由
3  """
4  from fastapi import APIRouter, Depends,
   HTTPException, status, Query
5  from pydantic import BaseModel
6  from typing import List, Optional
7  from app.interface.dependency import
   get_orders_handler
8  from
   app.application.user.queries.get_orders
   import GetOrdersQuery,
   GetOrdersHandler, OrderDto
9  from app.application.common.exception
   import DomainException
10
11
12  router = APIRouter(prefix="/orders",
   tags=["订单"])
13
14
15  class OrderResponse(BaseModel):
16      """订单响应"""
17      id: int
18      order_number: str
19      total_amount: float
20      status: str
21      created_at: str
22      updated_at: str
```

```
23
24
25 class GetOrdersResponse(BaseModel):
26     """获取订单响应"""
27     orders: List[OrderResponse]
28     total: int
29     limit: int
30     offset: int
31
32
33 @router.get("/",
34     response_model=GetOrdersResponse)
35 async def get_orders(
36     user_id: int = Query(...,
37     description="用户ID"),
38     limit: Optional[int] = Query(10,
39     ge=1, le=100, description="限制数量"),
40     offset: Optional[int] = Query(0,
41     ge=0, description="偏移量"),
42     handler: GetOrdersHandler =
43     Depends(get_orders_handler)
44 ):
45     """获取用户订单列表"""
46     try:
47         query = GetOrdersQuery(
48             user_id=user_id,
49             limit=limit,
50             offset=offset
51         )
52         result = await
53         handler.handle(query)
54
55
56
57
```

```
49         return GetOrdersResponse(  
50             orders=[  
51                 OrderResponse(  
52                     id=order.id,  
53  
54                     order_number=order.order_number,  
55  
56                     total_amount=order.total_amount,  
57  
58                     status=order.status,  
59  
60                     created_at=order.created_at,  
61  
62                     updated_at=order.updated_at  
63                     )  
64                 for order in  
65                 result.orders  
66             ],  
67             total=result.total,  
68             limit=limit or 10,  
69             offset=offset or 0  
70             )  
71     except DomainException as e:  
72         raise HTTPException(  
73  
74             status_code=status.HTTP_400_BAD_REQUEST,  
75  
76             detail=e.message  
77             )  
78     except Exception as e:  
79         raise HTTPException(  
80
```

```

72         status_code=status.HTTP_500_INTERNAL_S
           ERVER_ERROR,
73         detail="服务器内部错误"
74     )
75

```

- 实现API端点：
 - GET /: 获取订单列表
- 查询参数: user_id、limit、offset
- 分页支持: Query参数验证
- 数据转换: OrderDto转OrderResponse
- 异常处理: 统一错误响应

第六阶段：应用入口

目标: 启动应用和生命周期管理

16. 主应用文件

- **文件:** main.py
- **目的:** 应用程序入口和配置
- **具体开发内容:**

```

1 from contextlib import asynccontextmanager
2 from fastapi import FastAPI
3 from tortoise import Tortoise
4 from config.settings import settings
5 from app.interface.api.v1.auth_router
   import router as auth_router
6 from app.interface.api.v1.order_router
   import router as order_router

```

- 创建应用生命周期管理:

```

1 @asynccontextmanager
2 async def lifespan(app: FastAPI):
3     await init_database()
4     yield
5     await close_database()

```

- 数据库初始化函数:

```

1 async def init_database():
2     await Tortoise.init(
3         db_url=settings.db_url,
4         modules={'models':
5             ['app.infrastructure.database.orm_models']}]
6     await Tortoise.generate_schemas()

```

- 创建FastAPI应用:

```

1 def create_app() -> FastAPI:
2     app = FastAPI(
3         title=settings.app_name,
4         version=settings.app_version,
5         description="基于DDD架构的FastAPI
6         应用",
7         lifespan=lifespan
8     )
9     app.include_router(auth_router,
10        prefix="/api/v1")
11     app.include_router(order_router,
12        prefix="/api/v1")
13     return app

```

- 根路径端点: 返回应用信息和文档链接
- 运行配置: uvicorn服务器配置

运行代码

① 创建环境

```
1 uv init
2 uv venv -p 3.12 .venv
```

② 安装依赖

```
1 uv sync
```

③ 运行应用

```
1 uv run main.py
```

④ 访问文档: <http://127.0.0.1:8000/docs>